



**Ministry of High Education
Culture and Science city at Oct 6,
The High Institute of Computer Science &
Information Systems**



Graduation Project

**Agri AI: Intelligent Plant Disease Recognition System deep
learning**

Supervised by:

Dr. Abeer Zidan

Assistant:

TA. Martina Safwat

Project No: F26_106

Academic Year :2025/2026



**Ministry of High Education
Culture and Science city at Oct 6,
The High Institute of Computer Science &
Information Systems**



Graduation Project

Agri AI: Intelligent Plant Disease Recognition System deep learning

Submitted by:

#	ID	Name
1	223815	زياد نادر عبد الله احمد
2	221123	حسام محمد مرزوق شحاته
3	220897	حمدي عرفه حمدي سيد
4	223455	علي عادل محمد محمد جوده
5	212892	زياد محمد عبد الرحمن عبد السميع
6	211930	درويش عبد الحميد درويش
7	220887	عاصم خميس احمد عبد الله

**A graduation project submitted in partial fulfillment of the
requirements for the degree of Bachelor of Computer Science and
Information System.**

Supervised by:

Dr. Abeer Zidan

Assistant:

TA. Martina Safwat

Project No: F26_106

Academic Year: 2025/2026

Acknowledgement

I would like to express my sincere gratitude to **Culture and Science City and the Higher Institute for Computer Science and Information Systems** for providing the academic environment and support that contributed to the completion of this project.

I would also like to extend my sincere appreciation to Dean **Prof. Sherif El-Atreby** for his leadership and continuous efforts in supporting students and fostering an environment that encourages academic excellence and innovation.

I would like to express my gratitude to all faculty members of the Higher Institute for Computer Science and Information Systems for their dedication, knowledge, and valuable contributions to our academic and professional development throughout our studies.

I would like to extend my deepest thanks and appreciation to **Dr. Abeer Zidan** for her continuous guidance, valuable feedback, and academic support throughout the development of this project. Her insights and supervision played a crucial role in shaping the project's outcomes.

I am also sincerely grateful to **Eng. Martina Safwat** for her technical support, encouragement, and constructive guidance, which greatly helped in improving the practical implementation of this work.

Finally, I would like to acknowledge all researchers and developers in the field of deep learning and convolutional neural networks, as well as the open-source community whose tools and libraries made this project possible.

Abstract

Plant diseases are among the most critical factors affecting agricultural productivity and crop quality, causing substantial economic losses and threatening global food security. Early identification of plant diseases is essential for effective treatment and prevention; however, conventional disease Recognition methods depend largely on visual inspection by agricultural experts, which is often time-consuming, expensive, and not always accessible, especially in rural areas.

This project presents an automated plant disease Recognition and classification system using **Convolutional Neural Networks (CNNs)**, aiming to provide a reliable and efficient alternative to traditional diagnosis techniques. The proposed system leverages deep learning to analyze images of plant leaves and accurately identify disease types at early stages. A comprehensive dataset of labeled leaf images representing healthy plants and multiple disease categories is utilized to train the model.

Several preprocessing steps are applied to enhance model performance, including image resizing, normalization, and data augmentation techniques such as rotation and flipping to improve robustness and reduce overfitting. The CNN architecture is designed to automatically extract high-level features from raw images, eliminating the need for manual feature engineering. The model is trained and evaluated using standard evaluation metrics such as accuracy, loss, and validation performance.

Experimental results demonstrate that the developed CNN model achieves high accuracy in classifying plant diseases, confirming the effectiveness of deep learning approaches in agricultural image analysis. The system shows

Table of Contents

Acknowledgment.....	3
Abstract.....	4
Chapter1: Introduction	13
1.1 Overview	14
1.2 Problem Statement	15
1.3 Motivation	16
1.4 Objective	17
1.5 Aim	17
1.6 Scope	17
1.7 General Constraints	18
1.8 Organization of the Project.....	18
Chapter 2 Background and Previous work	19
2.1 Background.....	20
2.2 Previous work.....	21
Chapter 3 Planning and Analysis	23
3.1 Planning.....	24
3.1.1 Feasibility study and estimated cost.....	25
3.1.2 Gantt chart.....	26
3.2 Analysis and limitations of existing system.....	27
3.3 Need for new system.....	28
3.4 Analysis of new system.....	29
3.4.1 Introduction	29
3.4.2 Overall Description	30
3.4.3 Specific Requirements	31
3.4.4.1 Functional Requirements.....	31

3.4.4.2 Non- Functional Requirements	32
3.5 External Interface Requirements.....	33
Chapter 4: Design	34
4.1 Design and Implementation Constraints.....	35
4.2 Assumptions and dependencies.....	38
4.3 Risks and risk management.....	39
4.4 User Experience (UX)	41
4.4.1 Problem statement.....	42
4.4.2 Persona.....	43
4.4.3 Empathy map.....	44
4.4.4 User journey map.....	45
4.4.5 Information Architecture (IA).....	46
4.4.6 User Flow.....	47
4.4.7 Wireframes.....	48
4.4.7.1 Low-fidelity wireframes.....	49
4.4.7.2 High-fidelity wireframes.....	51
4.5 User Interface (UI).....	53
4.6 Design of database ERD.....	55
4.7 Class diagram.....	55
4.8 Use case diagram.....	56
4.9 Activity diagram.....	58
4.10 Sequence diagram.....	59
4.11 Context diagram.....	60

Chapter 5 Implementation.....	61
5.1 Software Architecture	62
5.2 User Interface.....	65
5.3 Results and Discussion.....	67
Chapter 6: Testing.....	70
6.1 Unit Testing.....	71
6.2 Integration Testing.....	72
Conclusion.....	74
Future work	75
References	76
Appendices	79
ملخص مشروع نظام التعرف على أمراض النبات	80

List of figures

Figure 3.1 (Gantt Chart)	26
Figure 4.1 (Problem statement)	42
Figure 4.2 (Persona)	43
Figure 4.3 (Empathy Map)	44
Figure 4.4 (User Journey Map)	45
Figure 4.5 (Information Architecture (IA))	46
Figure 4.6 (User Flow)	47
Figure 4.7 (Low-Fidelity Wireframes-Homepage)	50
Figure 4.8 (Low-Fidelity Wireframes- Upload Image page)	50
Figure 4.9 (Low-Fidelity Wireframes- Diagnosis Result page.....	50
Figure 4.10 (High-Fidelity Wireframes- Homepage)	52
Figure 4.11 (High-Fidelity Wireframes- Upload Image page)	52
Figure 4.12 (High-Fidelity Wireframes- Diagnosis Result page)	52
Figure 4.13 (Homepage UI)	54
Figure 4.14 (Upload Image page UI)	54
Figure 4.15 (Diagnosis Result page UI)	54
Figure 4.16 (Class Diagram)	55
Figure 4.17 (Use-case Diagram)	57

Figure 4.18 (Sub Use-case Diagram)	57
Figure 4.19 (Activity Diagram)	58
Figure 4.20 (Sequence Diagram)	59
Figure 4.21 (Context Diagram)	60
Figure 5.1: (Image Preprocessing)	62
Figure 5.2: (CNN Architecture)	63
Figure 5.3: (Model Deployment and Prediction Interface Implementation)	66
Figure 5.4: (Training and Validation Accuracy)	67
Figure 5.5: (Training and Validation Loss)	68
Figure 5.6: (Treatment Recommendation System)	69
Figure 6.1: (System Integration Workflow)	72
Figure 6.2: (Integrated System Output)	73

List of Tables

Table 3.1 (Estimated Cost)	26
Table 4.1 (Risk and Risk Management)	39
Table 6.1 (Unit Testing)	71
Table 6.2 (Integration Testing)	72

List of abbreviations

AI	Artificial Intelligence
ML	Machine Learning
CNN	Convolutional Neural Network
XAI	Explainable Artificial Intelligence
SRS	Software Requirements Specification
ERD	Entity Relationship Diagram
UX	User Experience
UI	User Interface
IA	Information Architecture
Lo-Fi	Low-Fidelity
Hi-Fi	High-Fidelity
GPU	Graphics Processing Unit
CPU	Central Processing Unit
Conv2D	2D Convolutional Layer

SVM	Support Vector Machine
KNN	K-Nearest Neighbors
LBP	Local Binary Pattern
JPEG	Joint Photographic Experts Group
PNG	Portable Network Graphics
JSON	JavaScript Object Notation
L.E	Egyptian Pound
ID	Identification Number
TA	Teaching Assistant
Prof	Professor
Dr	Doctor

Chapter 1

Introduction

1.1 Overview

Agriculture is considered one of the most essential sectors worldwide, as it plays a fundamental role in providing food, raw materials, and employment opportunities for a large portion of the global population. The agricultural sector significantly contributes to economic stability and food security, especially in developing countries. However, agricultural productivity faces many challenges, among which plant diseases are one of the most serious and damaging factors.



Plant diseases are a major threat to agricultural productivity, causing significant crop losses if not detected early. Traditional disease diagnosis relies on manual inspection and expert knowledge, which can be time-consuming, costly, and unavailable in many rural areas.

Recent advances in artificial intelligence, particularly deep learning, have provided effective solutions for automated disease detection. Convolutional Neural Networks (CNNs) have demonstrated high performance in image classification and recognition tasks.

1.2 Problem Statement

Plant disease Recognition using traditional manual methods presents several challenges that limit its effectiveness and accessibility. Farmers often rely on visual inspection and personal experience to identify plant diseases, which can lead to incorrect diagnoses due to similarities between disease symptoms or lack of sufficient expertise. In many cases, early symptoms of diseases are subtle and difficult to detect with the naked eye, causing delayed intervention.

Furthermore, access to agricultural experts is limited in many regions, especially in rural and developing areas. This lack of expert support results in late disease identification, which allows diseases to spread rapidly across crops. Consequently, farmers may resort to excessive or inappropriate use of pesticides, leading to increased costs, environmental pollution, and potential health risks.

Existing challenges can be summarized as follows:

- Manual disease Recognition is time-consuming and prone to human error.
- Accurate diagnosis requires expert knowledge that is not always available.
- Delayed Recognition leads to significant crop losses and reduced productivity.

1.3 Motivation

The motivation for this project stems from the increasing demand for smart agricultural solutions that leverage modern technologies to improve efficiency and productivity. With the rapid growth of the global population, the need for higher agricultural output has become more urgent than ever. Reducing crop losses caused by plant diseases is essential to meet this demand.

The success of deep learning models, particularly CNNs, in image classification tasks has inspired their application in agriculture. CNNs are capable of automatically extracting relevant features from images without manual feature engineering, making them suitable for analyzing complex visual patterns found in diseased plant leaves.

Another major motivating factor is the widespread availability of digital imaging devices such as smartphones and cameras. This availability makes it feasible to capture plant images easily and use automated systems for disease diagnosis. The development of an AI-based plant disease Recognition system can empower farmers with limited technical knowledge by providing them with a simple yet effective diagnostic tool.

1.4 Objective

The main objectives of this project are as follows:

- To study and analyze plant diseases and their visual symptoms on leaves.
- To design and implement a Convolutional Neural Network for plant disease classification.
- To preprocess plant leaf images to enhance model accuracy and performance.
- To train and evaluate the model using labeled datasets of healthy and diseased leaves.

1.5 Aim

The main goal of this project is to create an automated and intelligent plant disease recognition system that accurately classifies diseases from leaf images. It helps farmers and agricultural experts by enabling early detection, reducing crop losses, and improving agricultural productivity.

1.6 Scope

The scope of this project includes recognizing and classifying plant diseases using leaf images only. It focuses on applying supervised deep learning techniques, particularly CNNs, along with image preprocessing methods such as resizing, normalization, and augmentation. The project also includes evaluating the model's performance using accuracy and loss metrics.

1.7 General Constraints

Despite the advantages of the proposed system, certain constraints are identified:

- The accuracy of the model depends on the quality and diversity of the dataset.
- Variations in lighting conditions, background noise, and image resolution may affect results.
- High computational resources are required for training deep learning models.
- The system may not perform well on diseases or plant species not included in the training data.

1.8 Organization of the Project

This project is structured into the following chapters:

- **Chapter 1:** Introduction, problem statement, objectives, scope, and constraints.
- **Chapter 2 :**Background and Previous work
- **Chapter 3:** Planning And Analysis
- **Chapter 4:** Design
- **Chapter 5:** Implementation.
- **Chapter 6:** Testing

Chapter 2

Background and Previous work

2.1 Background

Plant diseases significantly reduce agricultural productivity and threaten global food security due to yield losses caused by fungi, bacteria, viruses, and nematodes. These diseases affect different parts of plants and often appear as visible symptoms such as spots, discoloration, wilting, and deformation. Traditionally, plant disease detection depends on manual inspection by experts, which can be accurate but is limited by availability, subjectivity, and difficulty in identifying early-stage infections, leading to delays and increased spread of disease.

To overcome these limitations, automated systems based on computer vision and machine learning have been developed to detect and classify plant diseases from images. These systems provide faster, more consistent, and scalable solutions compared to manual methods. Machine learning techniques map image features to disease categories, but they often require manual feature extraction such as color, texture, and shape, which can be time-consuming and dependent on expert knowledge.

Deep learning, especially Convolutional Neural Networks (CNNs), has greatly improved image classification by automatically learning features directly from raw images. CNNs detect patterns like edges, textures, and complex structures that help distinguish between healthy and diseased plants with high accuracy. The use of CNNs in plant disease recognition offers advantages such as automated feature extraction, improved accuracy, better generalization, and scalability across different crops and disease types.

2.2 Previous Work

Over the past decade, considerable research efforts have been made to develop automated plant disease Recognition systems. These efforts span multiple methodologies, datasets, model architectures, and performance evaluations. In this section, we present some of the most relevant and influential studies in the field.

Traditional Machine Learning Approaches

Early research in plant disease detection relied on traditional machine learning methods such as SVM, KNN, and Random Forests after extracting manual image features. These approaches used features like color, texture, and LBP from leaf images and achieved acceptable accuracy in some cases. However, their performance depended heavily on handcrafted features and extensive preprocessing, which limited their ability to generalize to different disease conditions.

Deep Learning Approaches

With the rise of deep learning, researchers started using CNN models to improve plant disease recognition. A key study by Mohanty et al. (2016) used AlexNet and GoogleNet on the PlantVillage dataset and achieved very high accuracy, showing the strength of deep learning in this field. Later studies, such as Too et al. (2019), compared different CNN architectures like VGGNet, ResNet, and Inception, and found that deeper models like ResNet

Hybrid and Transfer Learning Methods

Researchers have also used hybrid methods that combine deep learning with traditional techniques or apply transfer learning using pre-trained models like those trained on ImageNet. Ferentinos (2018) showed that transfer learning with CNNs can achieve high accuracy and reduce training time even with limited labeled data, proving that general image knowledge can be adapted for plant disease recognition.

Other studies also used image segmentation to focus on infected areas of leaves before classification. Although this can improve accuracy in some cases, it adds extra preprocessing steps and increases system complexity.

Limitations of Previous Work

Despite the progress and high accuracies reported in research studies, several limitations persist:

Many models are trained and evaluated on controlled datasets with uniform backgrounds, which do not reflect real-world field conditions.

Performance may degrade when tested on images captured under varying lighting, angles, and backgrounds.

Some approaches require high computational resources.

Chapter 3

Planning And

Analysis

3.1 Planning

The planning phase represents the foundation of the project development process. It focuses on identifying the required steps, resources, and timeline needed to build the plant disease Recognition system using Convolutional Neural Networks (CNN). This phase includes determining the feasibility of the project, estimating the cost, identifying potential risks, organizing the workflow, and designing a clear schedule for project execution. Proper planning ensures that every stage is aligned with the project objectives and that all tasks are carried out efficiently within a reasonable time frame and available resources.

The planning phase also includes defining team responsibilities, understanding the required dataset, selecting development tools, and specifying the evaluation criteria that will be used to measure the success of the system. This organized structure contributes to reducing errors, avoiding delays, and ensuring the successful completion of the project.

3.1.1 Feasibility Study and Estimated Cost

The feasibility study aims to determine whether the project can be executed effectively based on technical, economic, and operational factors. It confirms if the proposed system is achievable with the available tools, time, and team capabilities.

A. Technical Feasibility

- The project is technically feasible because it relies on Python, TensorFlow/Keras, CNN models, and publicly available datasets such as PlantVillage.
- Deep learning tools like Google Colab or Kaggle Notebooks can be used for training without the need for purchasing high-end hardware.
- Image preprocessing and dataset handling are manageable with libraries such as OpenCV, NumPy, and Pandas.

Conclusion: The project is technically achievable with current resources.

B. Estimated Cost

Item / Resource	Description / Usage	Estimated Cost
Laptop/PC (Existing)	System development & testing	0 L.E (Available)
Google Colab	Model training environment	0 L.E (Free Tier)

Item / Resource	Description / Usage	Estimated Cost
Python Libraries & Frameworks	TensorFlow, Keras, NumPy, OpenCV, etc.	0 L.E (Open-source)
Dataset	PlantVillage Dataset	0 L.E (Free)
Internet Usage / Power Consumption	Research & model training	~200 L.E
Documentation & Printing	Final report & presentation	~150 L.E

Table 3.1(Estimated Cost)

3.1.2 Gantt Chart (Project Timeline)



Figure 3.1(Gantt Chart)

3.2 Analysis and Limitations of Existing System

Current plant disease Recognition systems vary from manual visual inspection by experts to semi-automated software solutions. Despite their widespread use, existing systems present several limitations:

3.2.1 Manual Inspection

Traditional methods rely on expert knowledge to identify diseases by visual symptoms. Limitations include:

1. Time-consuming: Inspecting large fields manually requires substantial effort.
2. Inaccuracy: Human observation is prone to errors, especially in the early stages of disease.
3. Limited Access: Many rural areas lack sufficient agricultural specialists.

Semi-Automated and Machine Learning Systems

Earlier automated systems use image processing and classical machine learning models (e.g., SVM, KNN, Random Forest) to classify diseases.

Limitations include:

1. Dependence on handcrafted features (color, texture, shape), which may not capture subtle disease characteristics.
2. Low generalization to different lighting conditions, backgrounds, or leaf orientations.
3. Performance degradation on complex or noisy datasets.

3.3 Need for New System

The proposed system aims to overcome the shortcomings of existing solutions by providing an automated, scalable, and accurate plant disease Recognition platform. The main reasons for developing a new system include:

1. **Enhanced Accuracy and Reliability**

Deep learning-based models, particularly CNNs, can automatically extract high-level features from leaf images, improving the Recognition of subtle disease symptoms.

2. **Time and Resource Efficiency**

The automated system reduces the need for manual inspection, allowing farmers and experts to diagnose diseases quickly across large areas.

3. **Wider Accessibility**

By integrating the system into a web-based or mobile application, farmers in remote or rural areas can access accurate disease diagnostics without relying on experts.

4. **Adaptability and Scalability**

The system can be trained to include new plant species and disease types as more data becomes available, making it scalable for various crops and regions.

5. **Support for Sustainable Agriculture**

Accurate early Recognition reduces unnecessary pesticide use,

3.4 Analysis of New System

3.4.1 Introduction

Purpose

This document provides a detailed Software Requirements Specification (SRS) for **Plant Disease Recognition System**. . The purpose of this system is to automatically detect plant pests and diseases using intelligent image processing and machine learning techniques. This document is intended for software developers, system designers, project managers, and stakeholders involved in the development and deployment of the system.

Scope

The Intelligent Pest Plants Recognition System helps identify plant pests and diseases from uploaded images using trained models, providing recognition results and recommendation Definitions, Acronyms, and Abbreviations

- **SRS:** Software Requirements Specification
- **AI:** Artificial Intelligence
- **ML:** Machine Learning
- **CNN:** Convolutional Neural Network
- **User:** Any individual using the system (farmer, researcher, admin)

3.4.2 Overall Description

Product Perspective

The Intelligent Pest Plants Recognition System is a standalone web-based and/or mobile-based application. It integrates image acquisition, preprocessing, machine learning-based pest detection, and result visualization modules.

Product Functions

- Upload plant images for analysis
- Detect pests and plant diseases using AI models
- Display Recognition results with confidence levels
- Provide pest information and basic treatment suggestions

User Classes and Characteristics

- **Farmers:** Basic users with minimal technical knowledge
- **Agricultural Experts:** Users with domain expertise
- **System Administrator:** Responsible for system maintenance and management

Operating Environment

- Client-side: Web browser or Android/iOS mobile application
- Server-side: Cloud or local server
- AI Frameworks: TensorFlow / PyTorch

3.4.3 Specific Requirements

3.4.3.1 Functional Requirements

Image Upload

- The system shall allow users to upload plant images in common formats (JPEG, PNG).
- The system shall validate image size and format.

Pest Detection

- The system shall analyze uploaded images using machine learning models.
- The system shall identify pests or diseases present in the image.
- The system shall return Recognition results with confidence percentages.

Result Display

- The system shall display the detected pest name.
- The system shall give a brief description and possible treatment methods.

3.4.3.2 Non-Functional Requirements

Performance Requirements

- The system shall process each image within an acceptable response time (less than 5 seconds).

Security Requirements

- The system shall protect user data and uploaded images.

Usability Requirements

- The system shall provide a simple and intuitive user interface.

Reliability Requirements

- The system shall be available 99% of the time.
- The system shall handle system failures gracefully.

Scalability Requirements

- The system shall support increasing numbers of users and image uploads.

3.4.4 External Interface Requirements

User Interface

- Responsive graphical user interface
- Simple upload and result display screens

Hardware Interfaces

- Camera (mobile devices)
- Server GPU (optional for faster processing)

Software Interfaces

- AI libraries (TensorFlow / PyTorch)

Chapter 4

Design

4.1 Design and Implementation Constraints

The design and implementation of the proposed Plant Disease Recognition System are subject to several technical, operational, and environmental constraints that must be considered to ensure successful development and deployment.

1. Model and Algorithm Constraints

- The system must rely on a Convolutional Neural Network (CNN) architecture for disease Recognition and classification.
- The CNN model architecture and training process should be documented and remain consistent unless retraining or improvement is explicitly planned.
- The model performance is constrained by the quality, size, and diversity of the training dataset.

2. Dataset Constraints

- The system will use pre-collected and publicly available datasets (e.g., PlantVillage).
- The dataset may not cover all plant species or disease variations found in real-field environments.
- Any imbalance in the dataset (unequal number of samples per class) may affect classification accuracy.

3. Image Input Constraints

- Uploaded images must be in supported formats such as JPEG or PNG.
- Image size and resolution must fall within predefined limits (e.g., maximum file size and minimum resolution) to ensure proper processing.
- Poor-quality images (blurred, low lighting, occluded leaves) may lead to reduced Recognition accuracy.

4. Hardware and Computational Constraints

- Model training may require significant computational resources, including GPU acceleration for faster convergence.
- Inference (prediction) should be optimized to run efficiently on standard CPU-based systems without requiring specialized hardware.
- The system must operate within the limitations of available memory and processing power, especially if deployed on low-cost devices.

5. Software and Platform Constraints

- The system uses open-source technologies such as Python, TensorFlow/Keras, OpenCV, and Flask/FastAPI.
- It is compatible with common operating systems and modern web browsers.

6. Performance Constraints

- The system should provide diagnosis results within an acceptable response time (e.g., ≤ 5 seconds per image on CPU).
- The system must handle single-user and limited multi-user requests without significant performance degradation.

7. Security and Privacy Constraints

- The system must not store or expose sensitive personal user data.
- Uploaded images and diagnosis results should be handled securely, especially if the system is deployed online.
- Unauthorized access to the system or its backend services must be prevented.

8. Deployment Constraints

- If deployed as a web-based application, the system must run on a standard web server without requiring specialized infrastructure.
- Internet connectivity is required for online deployment; offline functionality may be limited to local installations.

9. Maintenance and Extensibility Constraints

- The system design must be modular to allow future updates, such as adding new diseases, plants, or improving the CNN model.
- Any extension of the system may require additional data, retraining, and validation of the model.

4.2 Assumptions and Dependencies

The design and implementation of the Plant Disease Recognition System are based on a set of assumptions and dependencies that affect system functionality and performance.

Assumptions

- Users are assumed to provide clear and properly captured images of plant leaves.
- The uploaded images are expected to show visible disease symptoms in case of infection.
- The dataset used for training is assumed to be representative of the target plant species and diseases.
- Users have access to a device capable of capturing or uploading images.
- For web-based deployment, users are assumed to have a stable internet connection.

Dependencies

- The system depends on a trained Convolutional Neural Network (CNN) model for disease Recognition and classification.
- The implementation relies on external libraries and frameworks such as Python, TensorFlow/Keras, and OpenCV.

4.3. Risk and Risk Management

This project involves several potential risks that may affect system performance, development timeline, or usability. Risk management is applied to identify these risks early and define strategies to reduce their impact.

Risk	Description	Impact	Mitigation Strategy
Low Image Quality	Users may upload blurred or poorly lit images	Reduced diagnosis accuracy	Input validation, user instructions, image preprocessing
Dataset Limitations	The system relies on a pre-collected dataset of plant leaf images, often captured in controlled environments. These “clean” images may	Limited generalization, reduced performance in field conditions	Data augmentation, testing with real-field images, future dataset expansion

Risk	Description	Impact	Mitigation Strategy
	not represent real-world variability (lighting, backgrounds, leaf orientations, partial occlusions). Also, some plant species or rare diseases may not be available in the dataset		
Overfitting	Models may perform well on training data but poorly on new images	Poor real-world performance	Regularization techniques, validation, early stopping

Risk	Description	Impact	Mitigation Strategy
High Training Time	CNN training may require long computation time	Development delays	Use pre-trained models, cloud platforms (Colab/Kaggle)

Table 4.1(Risk and Risk Management)

4.4 User Experience (UX)

User Experience (UX) in the Plant Disease Recognition System focuses on providing an intuitive and efficient interaction for users. The system allows users to identify plant diseases easily by uploading a plant leaf image and receiving instant diagnostic results. The design minimizes user effort by reducing the interaction steps and presenting clear, direct functionality.

The interface emphasizes simplicity and visual clarity through a clean layout and nature-inspired colors. Key elements such as the image upload area and diagnosis results are clearly highlighted, ensuring ease of use. Overall, the UX design aims

4.4.1 Problem Statement

This is a Problem Statement for a plant disease recognition system project. The text describes the current challenges faced by farmers and non-specialized users in identifying plant diseases accurately. Traditional methods rely heavily on visual inspection and expert knowledge, which are often time-consuming, subjective, and not always available. In addition, many existing digital solutions suffer from complex user interfaces, making them difficult to use efficiently.

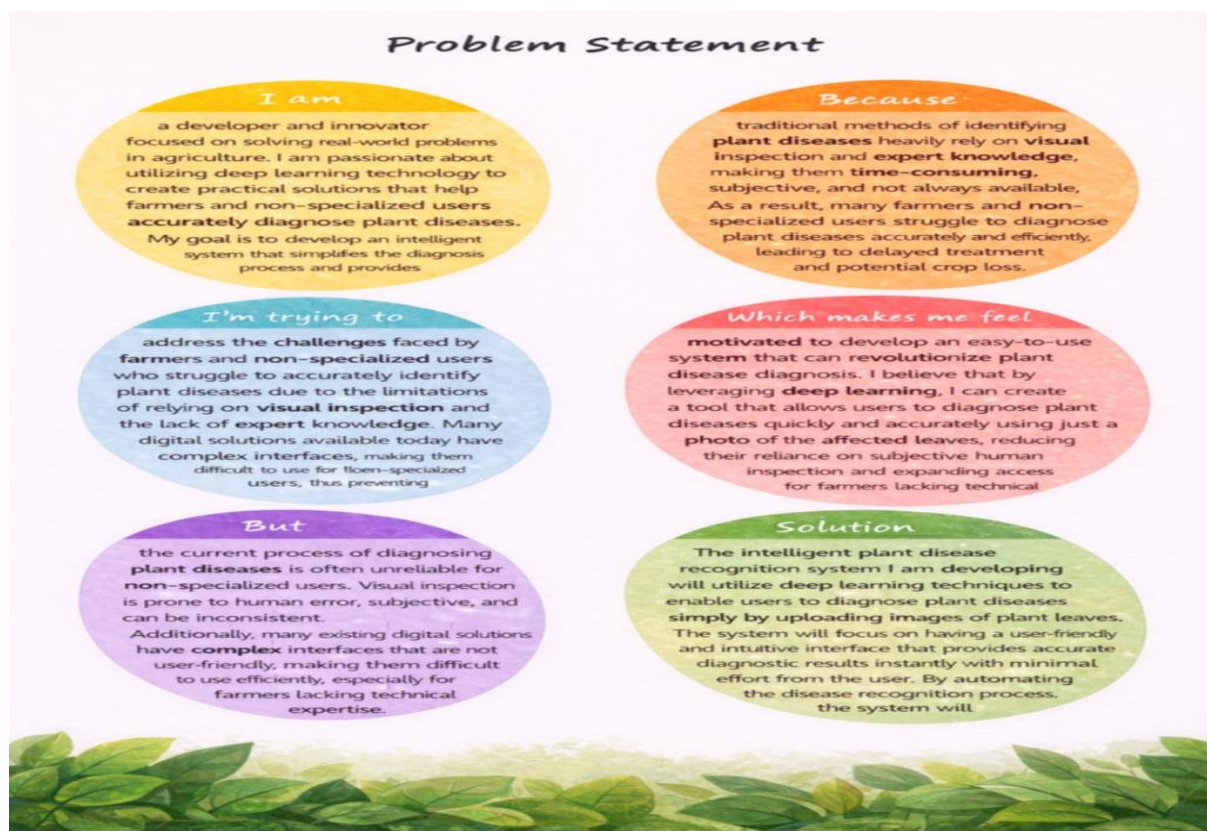


Figure 4.1 (Problem statement)

4.4.2 Persona

This is a User Persona for a farmer or non-specialized user who faces difficulties in identifying plant diseases accurately. The user relies on traditional methods and lacks expert knowledge, which leads to delays and uncertainty in diagnosis.

The user's goal is to obtain quick and accurate results through a simple and clear interface. The system is designed to minimize complexity by allowing users to upload a plant leaf image and receive immediate diagnostic feedback. This persona supports the design of a user-friendly interface that meets the needs of users with minimal technical experience.

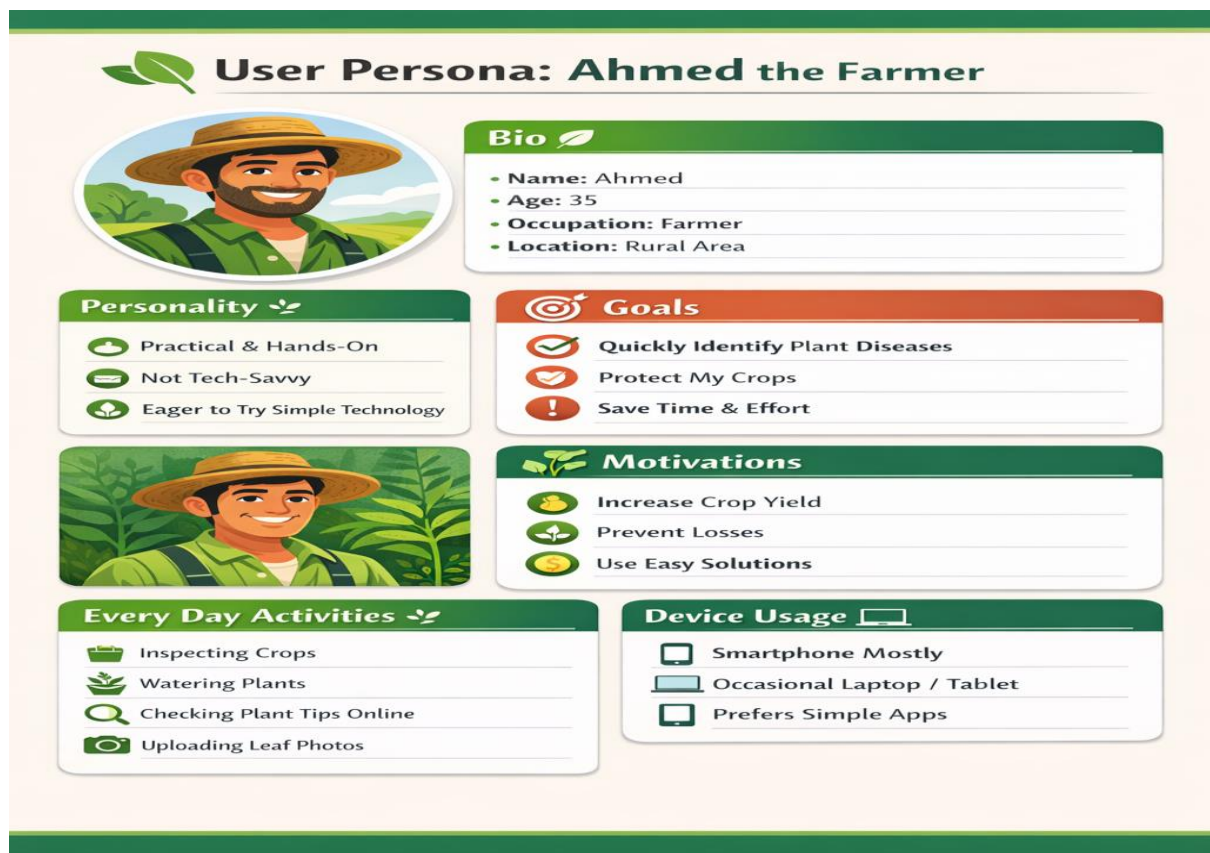


Figure 4.2 (Persona)

4.4.3 Empathy Map

This is an Empathy Map for a farmer using the Plant Disease Recognition System. It outlines the user's thoughts, feelings, actions, and concerns regarding plant disease diagnosis. The map highlights frustrations with traditional manual methods, the need for simple and immediate diagnostic tools, and the desire to protect crops efficiently. By understanding the user's emotions and behaviors, the system is designed to provide clear, intuitive, and actionable feedback, enhancing the overall user experience.

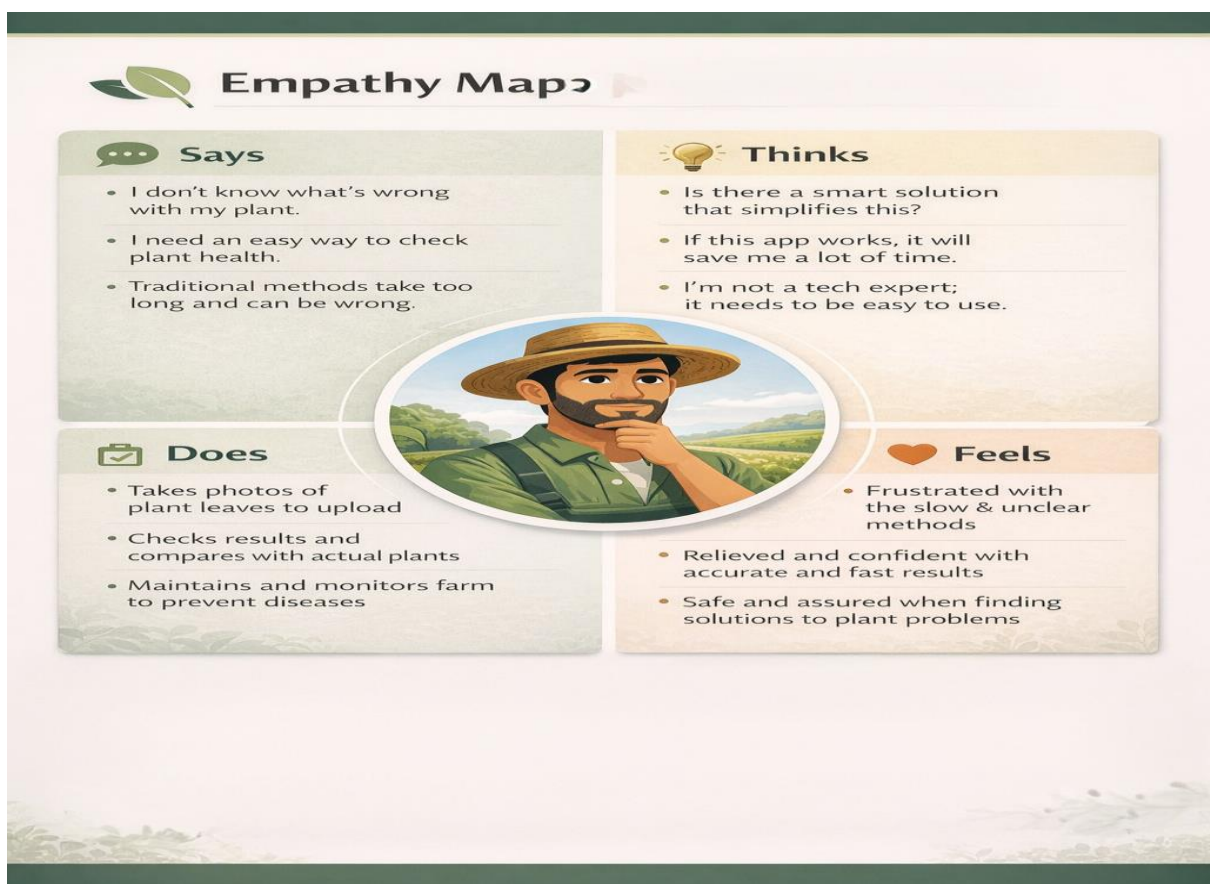


Figure 4.3 (Empathy Map)

4.4.4 User Journey Map

This is a User Journey Map for a farmer using the Plant Disease Recognition System. It outlines the steps the user goes through from planning which plants to check, capturing images, analyzing results, making treatment decisions, to searching for additional solutions. The map highlights challenges such as uncertainty in diagnosing plant diseases, the need for accurate and fast feedback, and the emotional responses at each stage. By understanding the user's journey, the system is designed to provide clear guidance, reduce stress, and support informed decision-making for effective crop management.

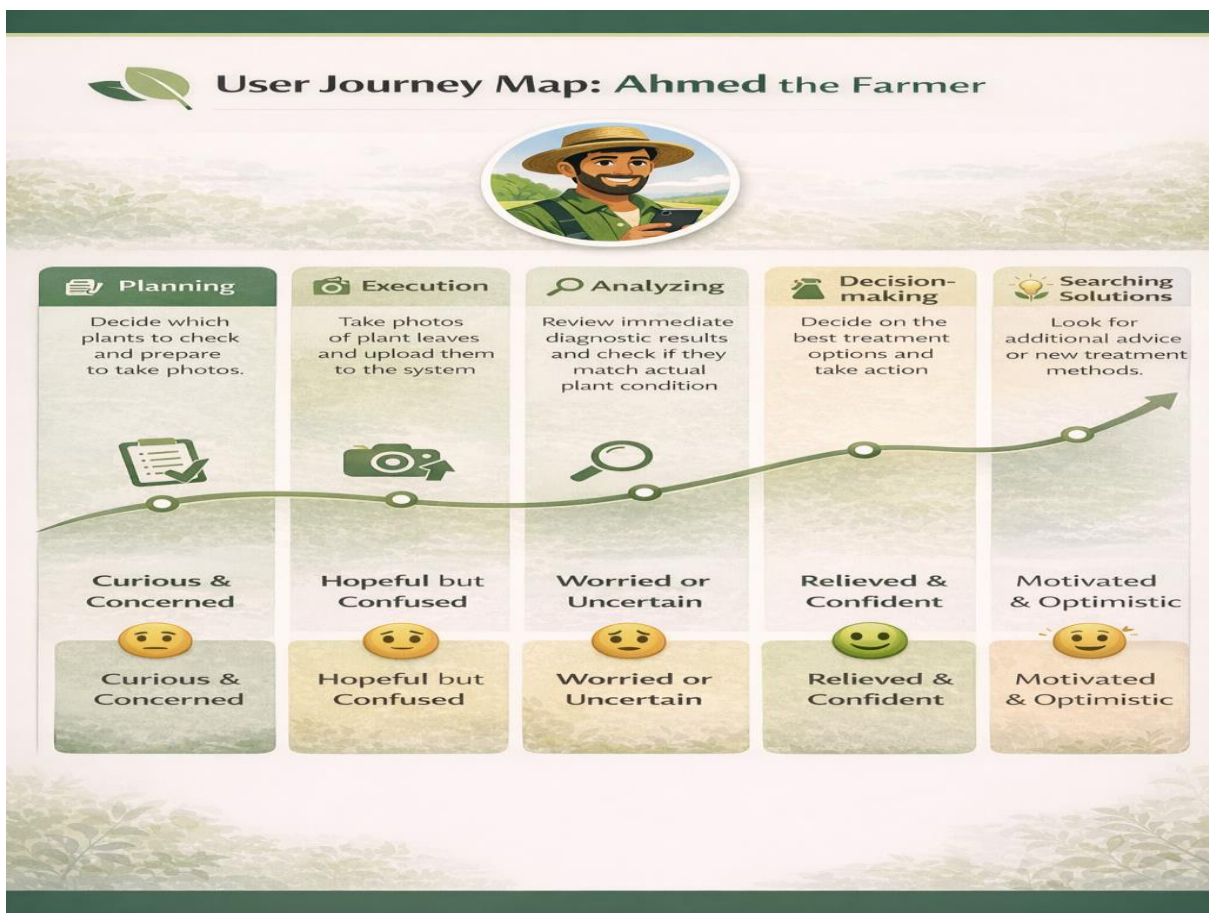


Figure 4.4 (User Journey Map)

4.4.5 Information Architecture (IA)

This is the Information Architecture of the Plant Disease Recognition System. It starts from the **Home Page**, leading to various modules such as **Upload Image and Plant Species Database**

Each section contains specific functionalities, for example:

- **Upload Image** allows users to submit plant leaf images for instant disease detection.
- **Plant Species Database** provides information about different plants and common diseases.

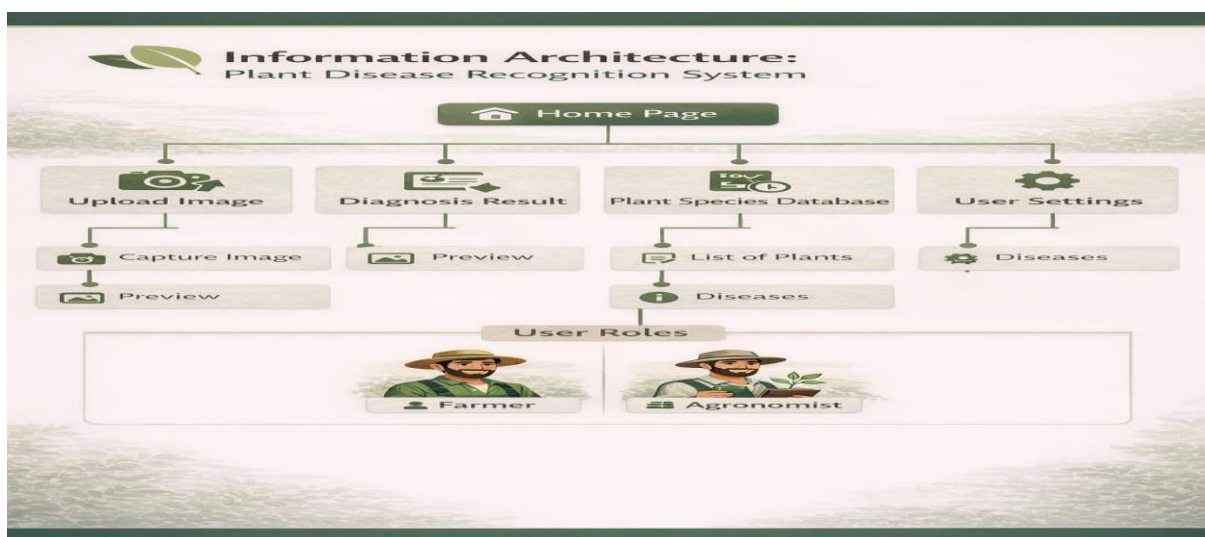


Figure 4.5 (Information Architecture (IA))

4.4.6 User Flow

The user flow begins when the user opens the web application and uploads an image of a plant leaf. The system then processes the image using the trained deep learning model and displays the diagnosis result, including the detected disease and confidence score. Users can also browse the plant species database to explore different plants and their associated diseases.

Flow Steps:

1. Open the Plant Disease Recognition System website.
2. Upload an image of the plant leaf.
3. System predicts disease and displays results:
 - Disease Name
 - Confidence
 - Related Plant Information

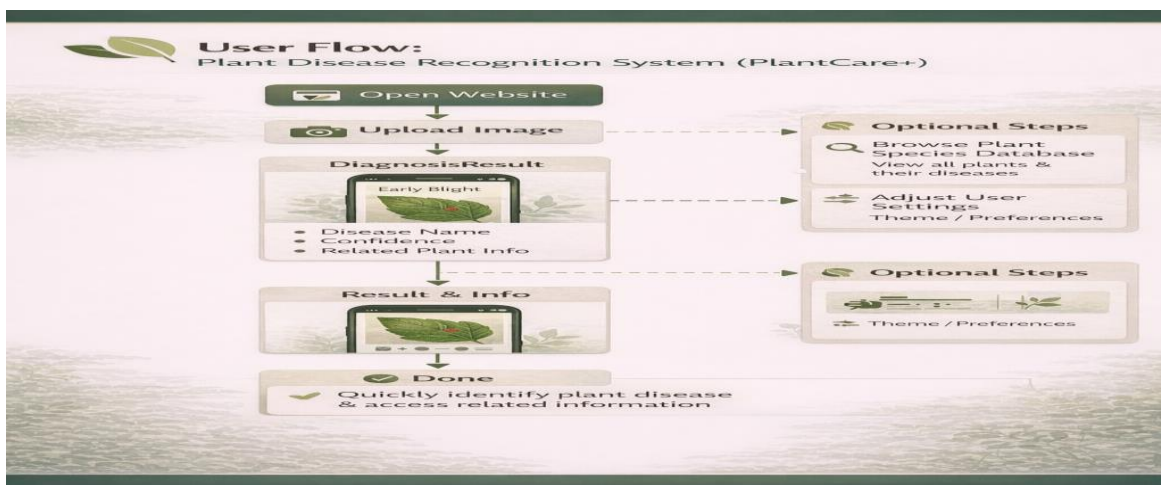


Figure 4.6 (User Flow)

4.4.7 Wireframes

Wireframes for the Plant Disease Recognition System serve as visual blueprints of the application's structure and layout. They focus on functionality and placement of content, without emphasizing colors, fonts, or visual styling.

The main wireframes include:

1. Home Page Wireframe

- **Header:** Project title and navigation links.
- **Main Section:** Hero section with a brief description and call-to-action to upload an image.
- **Footer:** Basic footer with copyright or project info.

2. Upload Image Wireframe

- **Upload Button:** Allows users to select a plant leaf image.
- **Preview Area:** Displays the uploaded image.
- **Instruction Text:** Guides the user on acceptable image formats and size.

3. Diagnosis Result Wireframe

- **Result Card:** Displays predicted disease name, confidence score, and related plant information.

4.4.7.1 Low-Fidelity Wireframes

Low-fidelity (Lo-Fi) wireframes are simple, sketch-like representations of the Plant Disease Recognition System layout. They focus on **structure, functionality, and user flow** rather than visual design details. Typically, they use **boxes, lines, and placeholders** for text, buttons, and images.

How They Are Created for this Project:

1. **Hand-drawn sketches** – Quickly drafted on paper or whiteboards to outline the main screens:
 - **Home Page** with Hero section and Upload Image button.
 - **Upload Image Page** showing the image preview area.
 - **Diagnosis Result Page** displaying disease name and confidence.
2. **Basic grayscale design** – Avoids colors, fonts, or real images, focusing only on **layout, navigation, and result placement**.

Lo-Fi wireframes are ideal for brainstorming, gathering initial feedback, and making quick changes to the **Plant Disease Recognition System interface** before moving to high-fidelity designs.

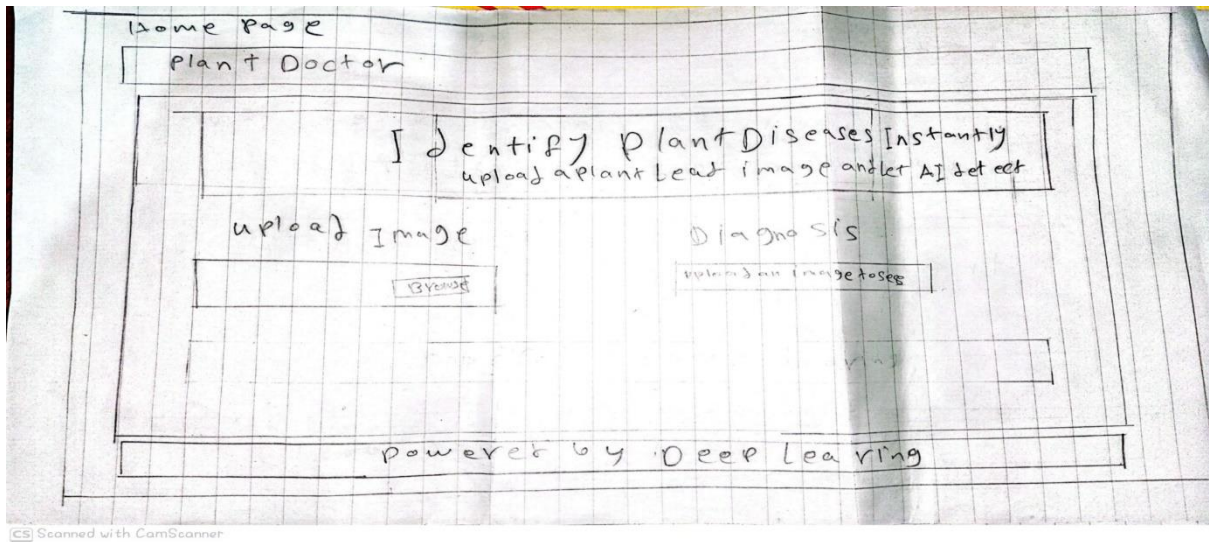


Figure 4.7(Low-Fidelity Wireframes-Homepage)

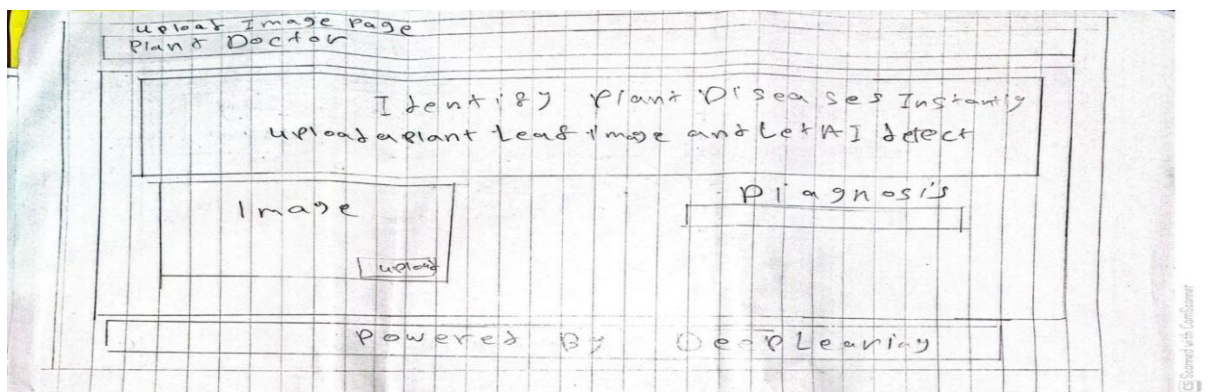


Figure 4.8 (Low-Fidelity Wireframes- Upload Image page)

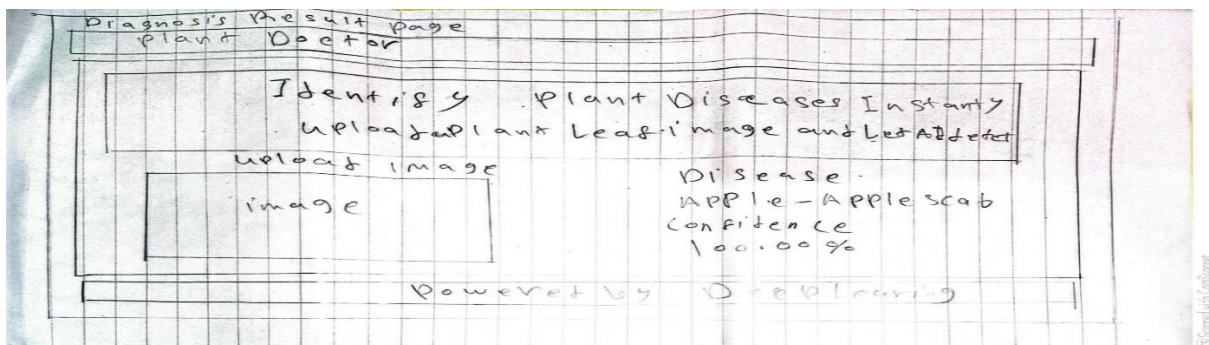


Figure 4.9 (Low-Fidelity Wireframes- Diagnosis Result page)

4.4.7.2 High-Fidelity Wireframes

High-fidelity (Hi-Fi) wireframes for the Plant Disease Recognition System are detailed, visually refined, and closely resemble the final application. They use **real UI elements, images, text, and buttons**, giving an accurate representation of how the user will interact with the system.

How They Are Created for this Project:

1. **Using design software** – Tools like Figma, Adobe XD, or Sketch can be used to recreate the Stream lit interface of the Plant Disease Recognition System.
2. **Incorporating real UI elements** –
 - **Home Page / Hero Section:** Includes the actual project title, description, and “Upload Image” button.
 - **Upload Image Page:** Shows the uploaded image in real dimensions, with interactive upload functionality.
 - **Diagnosis Result Page:** Displays the predicted disease name, confidence score, and related plant information exactly as it would appear in the app.

High-fidelity wireframes for this project are essential for:

- **Presentations** to stakeholders, showing exactly how the system looks and works.

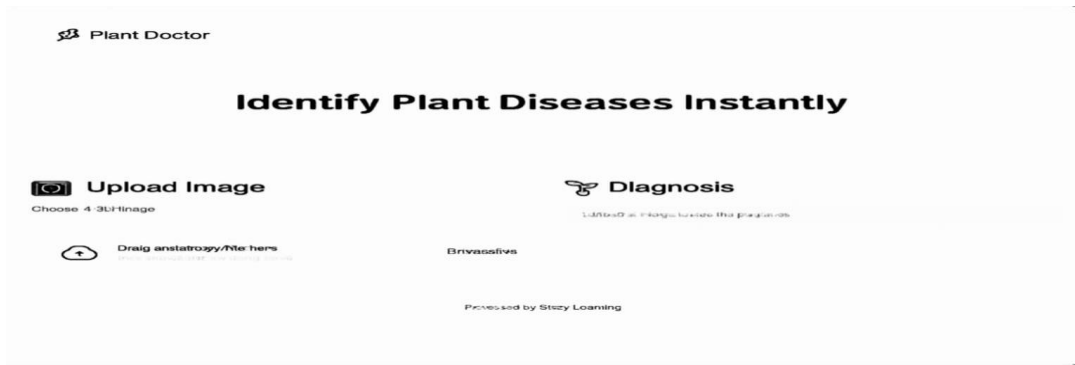


Figure 4.10 (High-Fidelity Wireframes- Homepage)

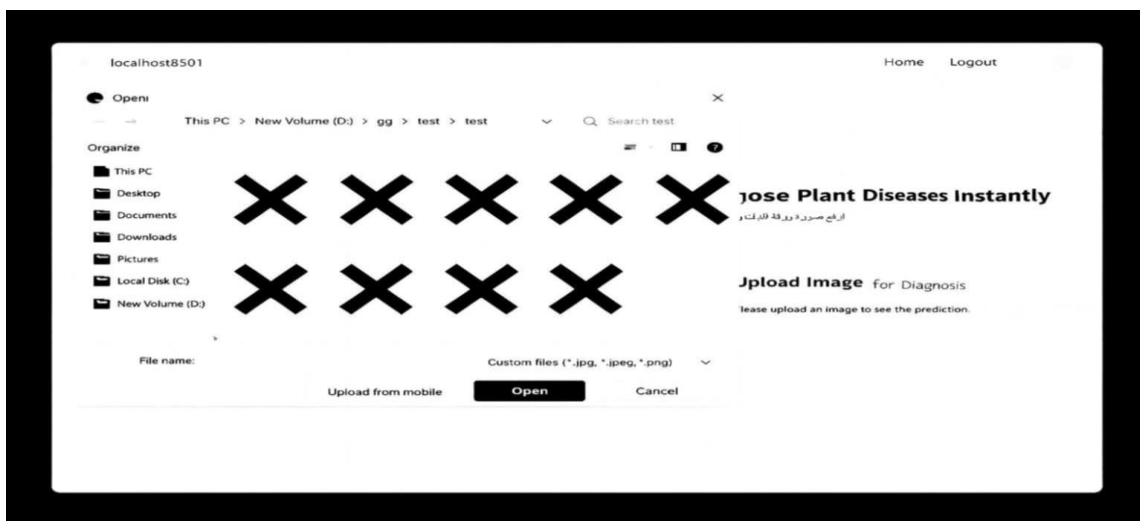


Figure 4.11 (High-Fidelity Wireframes- Upload Image page)



Figure 4.12 (High-Fidelity Wireframes- Diagnosis Result page)

4.5 User Interface (UI)

The User Interface (UI) is the visual part of the system that users interact with, including buttons, image upload areas, and result displays.

It is designed to be simple and clear so users can easily upload plant leaf images and view disease predictions. A clean UI improves usability and makes the system easier to use.

Steps to Design the UI in Figma for This Project:

- **Create a New Project** → A new Figma file is created to design the web-based interface of the Plant Disease Recognition System.
- **Add Screens (Frames)** → Web frames are added to represent the main screens of the system, such as the Home Page, Image Upload Screen, and Diagnosis Result Screen.
- **Insert Elements** → UI elements like the upload image button, image preview area, disease name text, confidence score, and navigation components are added based on the actual system workflow.
- **Set Colors and Fonts** → A consistent color scheme inspired by nature (green and white tones) is applied, along with clear and readable fonts to enhance accessibility and visual clarity.
- **Create Reusable Components** → Reusable components such as buttons and result cards are designed to maintain consistency across the interface and allow easy updates during development.



Figure 4.13 (Homepage UI)

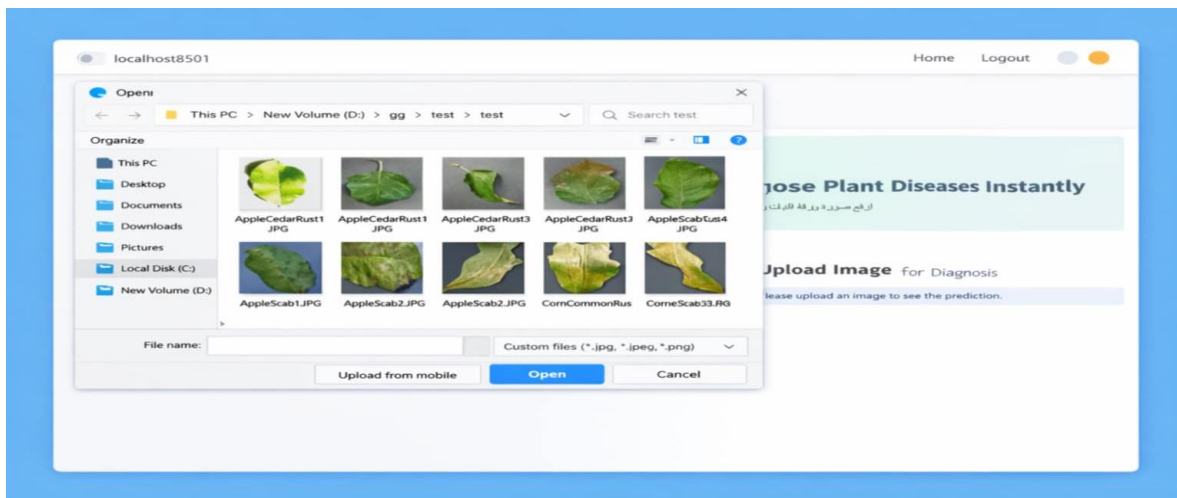


Figure 4.14 (Upload Image page UI)

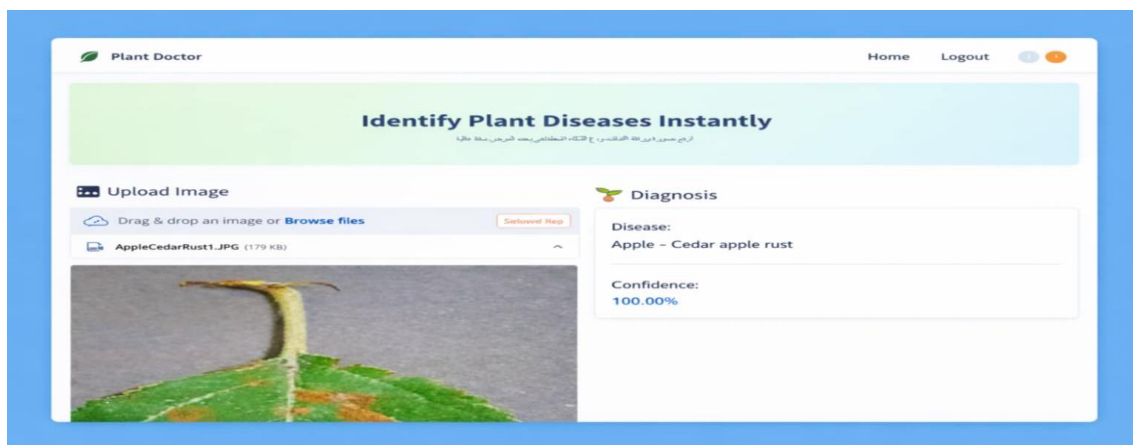


Figure 4.15 (Diagnosis Result page UI)

4.6 Design of Database / ERD

During the design of the Plant Disease Recognition System, it was determined that a database was not required. The system is designed to process uploaded plant leaf images and generate diagnosis results immediately using a CNN model. These results are displayed directly to the user and are not intended to be stored for future retrieval.

Since the system focuses on real-time disease Recognition rather than historical data analysis or user record management, introducing a database would add unnecessary complexity without providing significant value. Therefore, no database schema, Entity Relationship Diagram (ERD), or database mapping was implemented in this project.

4.7 Class Diagram of the Plant Disease Recognition System

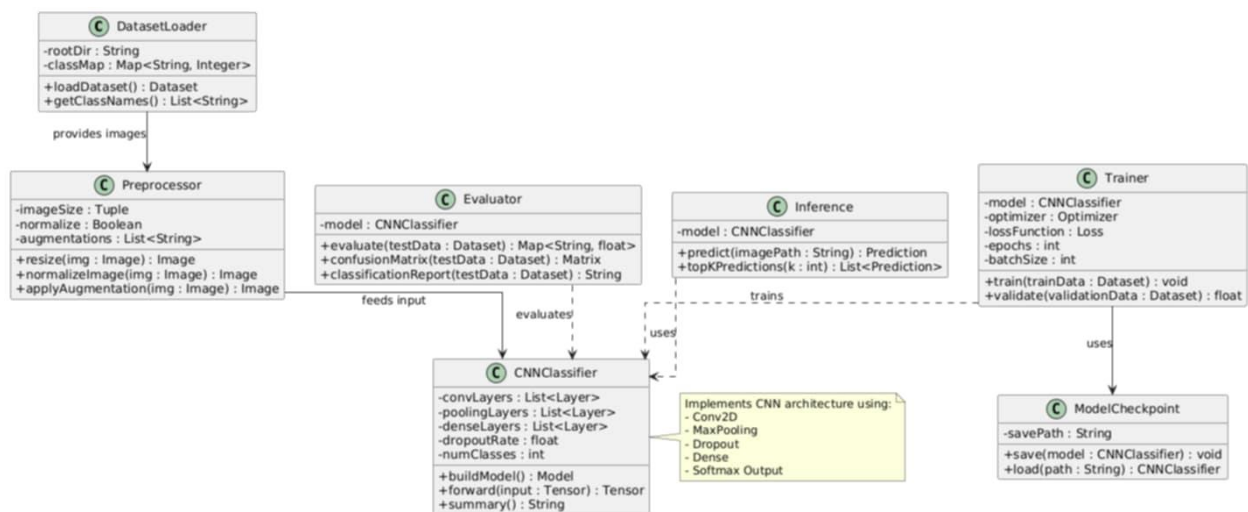


Figure 4.16(Class Diagram)

The diagram represents the overall structure and flow of the Plant Disease Recognition System for documentation purposes. It shows the main components and their interactions, including:

- 1. User Interface** – where users upload plant leaf images.
- 2. Image Preprocessing Module** – handles resizing, normalization, and preparation of images for the model.
- 3. CNN Model Module**– the core component that performs feature extraction and disease classification.
- 4. Result Modul** – generates and displays the diagnosis result with the disease name and confidence score.

4.8 Use-case Diagram of the Plant Disease Recognition System

The diagram illustrates the primary interactions between the user and the system. The main use-case represents the process of uploading a plant leaf image, which is then processed by the CNN model to generate a diagnosis.

Optional actions, such as retaking or re-uploading an image, are also included to allow users to correct mistakes or submit new samples. This diagram provides a clear conceptual overview of the system's functionality from the user's perspective, highlighting the key operations without detailing internal processing steps.

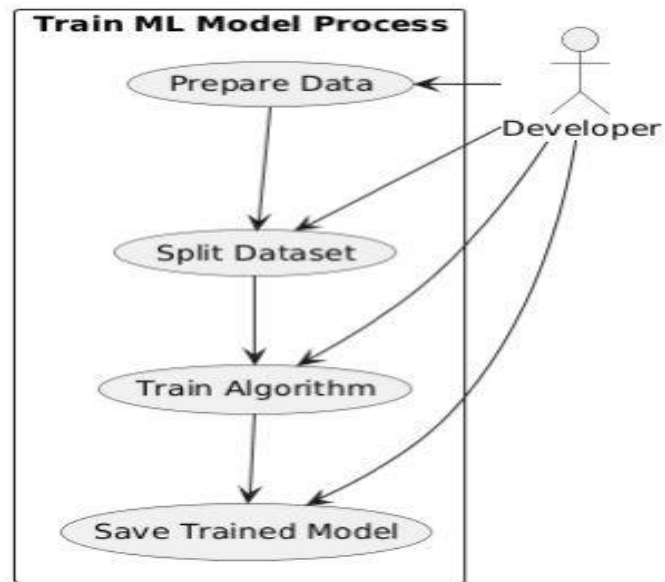


Figure 4.17(Use-case Diagram)

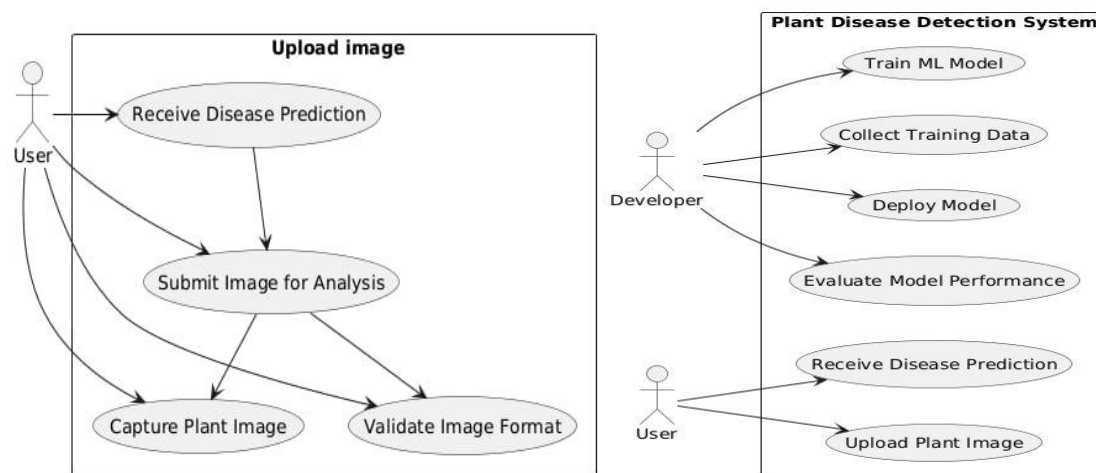


Figure 4.18(sub-Use-case Diagram)

4.9 Activity Diagram of the Plant Disease Recognition System

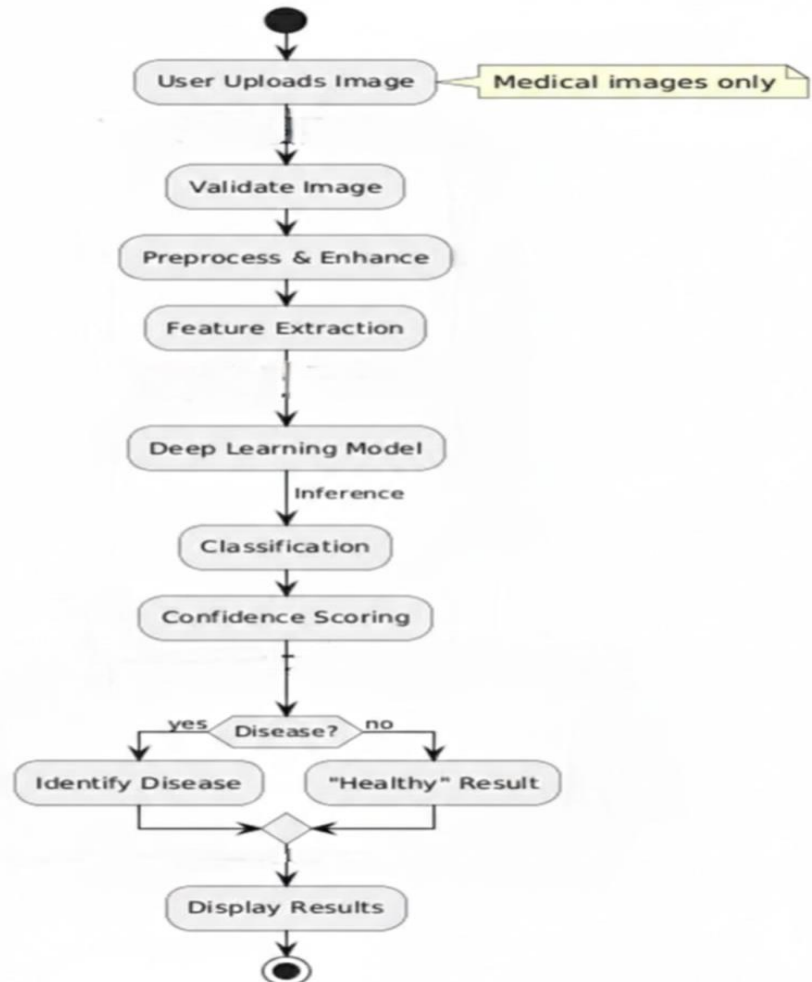


Figure 4.19(Activity Diagram)

The diagram illustrates the flow of operations within the system from the user's perspective. It begins with the user uploading a plant leaf image, followed by image preprocessing and processing through the CNN model for disease classification.

The system then generates and displays the diagnosis result, indicating whether the leaf is healthy or diseased, along with the confidence score. An optional step allows the user to retake or upload a new image if needed.

4.10 Sequence Diagram of the Plant Disease Recognition System

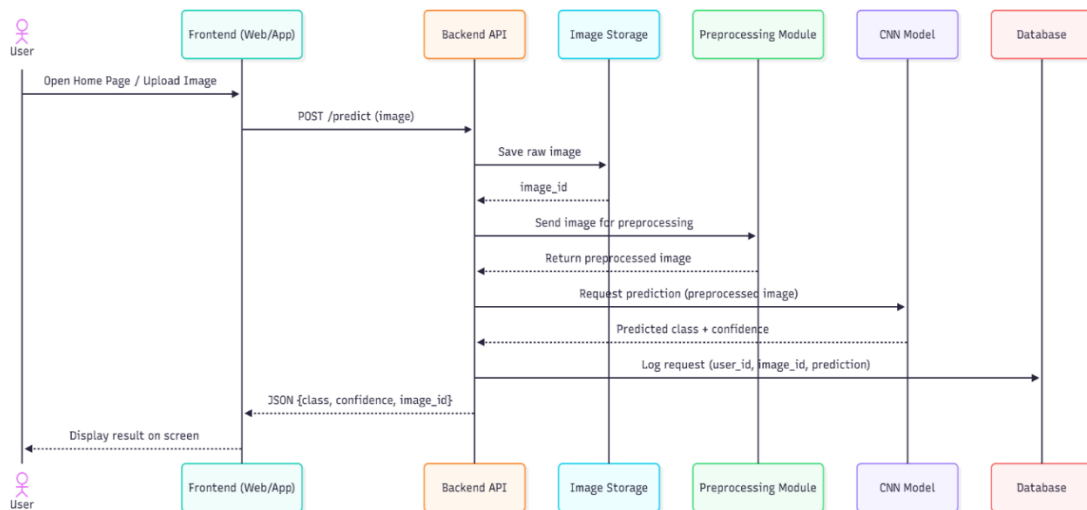


Figure 4.20(Sequence Diagram)

The sequence diagram illustrates the step-by-step interactions between the user and the system components. The process starts when the user uploads a plant leaf

4.11 Context Diagram of the Plant Disease Recognition System

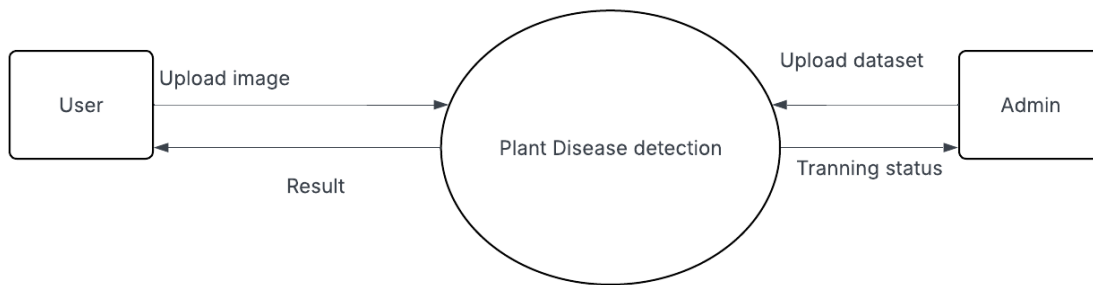


Figure 4.21(Context Diagram)

The figure illustrates a high-level system context diagram for a **Plant Disease Detection System**. The system is represented as a central processing entity that interacts with two primary external actors: the **User** and the **Admin**.

On the left side, the *User* interacts with the system by uploading an image of a plant leaf or crop. This image serves as the input data for analysis. After processing the image using the embedded detection model, the system returns the **result** to the user, which typically includes the identified disease classification and related diagnostic information.

Chapter 5

Implementation

5.1 Software Architecture

Software implementation is the process of developing and integrating all software components required for the Plant Disease Recognition System. The implementation includes model training, image preprocessing, prediction handling, and frontend integration.

Image Preprocessing

Image preprocessing is an essential step before prediction to prepare the input image for the CNN model. It includes resizing the image to 256×256 pixels, converting it into a NumPy array, normalizing pixel values by dividing by 255.0, and expanding its dimensions before feeding it into the model. These steps help ensure proper input formatting and improve prediction accuracy.

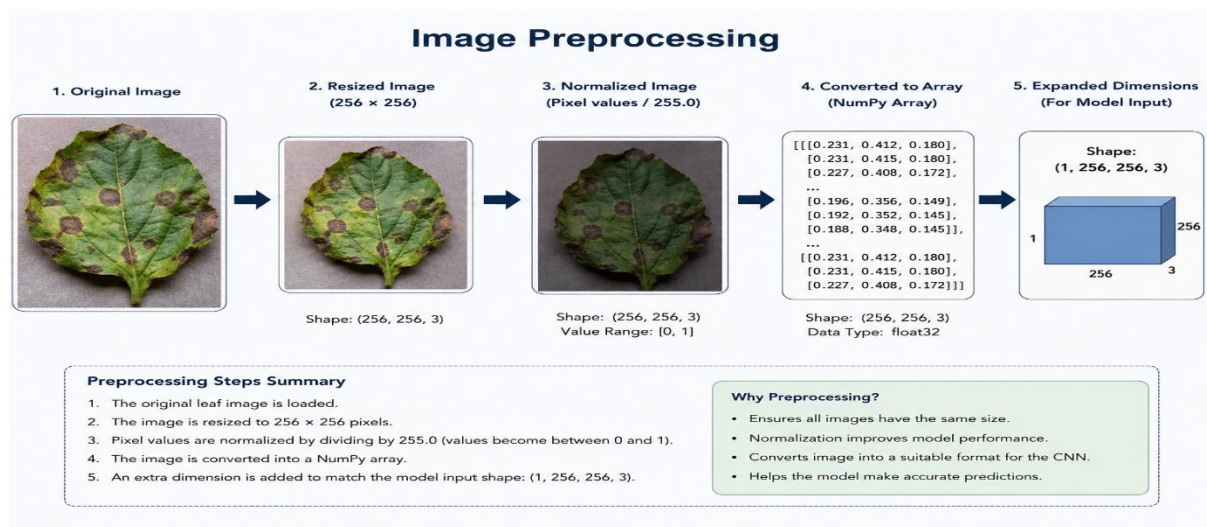


Figure 5.1: (Image Preprocessing)

CNN Architecture

The system uses a Convolutional Neural Network (CNN) for plant disease classification. The model extracts important visual features from leaf images, detects disease patterns such as spots and color changes, and classifies the image into the correct disease category.

The architecture consists of convolutional layers for feature extraction, pooling layers for reducing dimensions, and dense layers for final classification.

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 256, 256, 32)	896
conv2d_1 (Conv2D)	(None, 256, 256, 32)	9,248
max_pooling2d (MaxPooling2D)	(None, 85, 85, 32)	0
conv2d_2 (Conv2D)	(None, 85, 85, 64)	18,496
conv2d_3 (Conv2D)	(None, 85, 85, 64)	36,928
max_pooling2d_1 (MaxPooling2D)	(None, 28, 28, 64)	0
conv2d_4 (Conv2D)	(None, 28, 28, 128)	73,856
conv2d_5 (Conv2D)	(None, 28, 28, 128)	147,584
max_pooling2d_2 (MaxPooling2D)	(None, 9, 9, 128)	0
conv2d_6 (Conv2D)	(None, 9, 9, 256)	295,168
conv2d_7 (Conv2D)	(None, 9, 9, 256)	590,080
conv2d_8 (Conv2D)	(None, 9, 9, 512)	3,277,312
conv2d_9 (Conv2D)	(None, 9, 9, 512)	6,554,112
flatten (Flatten)	(None, 41472)	0
dense (Dense)	(None, 1568)	65,029,664
dropout (Dropout)	(None, 1568)	0
dense_1 (Dense)	(None, 38)	59,622

Total params: 76,092,966 (290.27 MB)

Trainable params: 76,092,966 (290.27 MB)

Non-trainable params: 0 (0.00 B)

Figure 5.2: (CNN Architecture)

Disease Categories

The model supports multiple plant species and diseases, including:

- o Apple Diseases
- o Tomato Diseases
- o Potato Diseases
- o Corn Diseases
- o Grape Diseases
- o Pepper Diseases
- o Strawberry Diseases
- o Healthy Plant Detection

The system can classify healthy and diseased leaves across different crop categories.

5.2 User Interface

The frontend was developed using Streamlit to provide a simple and interactive interface for uploading plant leaf images and viewing prediction results. It displays the detected disease, confidence score, and recommendations, while offering a user-friendly design with image preview and clear result presentation.

System Visualization

This section presents the implementation of the Plant Disease Recognition System using a Streamlit-based interface. The figure illustrates the deployed application, including the process of loading the trained convolutional neural network model, preprocessing input images, and generating predictions in real time.

The system integrates the inference pipeline with a user-friendly interface, allowing users to upload plant leaf images and receive classification results along with confidence scores. In addition, the system retrieves relevant disease information, including descriptions, treatments, and prevention guidelines from an external knowledge base.

This implementation demonstrates the end-to-end workflow of the model from input acquisition to final prediction and recommendation generation.

```

D:\saved_models > cd D:\snp
1
2 import streamlit as st
3 from tensorflow.keras.models import load_model
4 from PIL import Image
5 import numpy as np
6 import time
7 # إعدادات الصفحة
8 st.set_page_config(
9     page_title="Plant Disease Recognition System",
10     page_icon="🌿",
11     layout="wide"
12 )
13 # تحميل النموذج
14 with open("D:\saved_models\treat.json", "r", encoding="utf-8") as f:
15     treatments = json.load(f)
16 # CSS
17 css = """
18 <style>
19 body {
20     background-color: #f0f0f0;
21     font-family: 'Segoe UI', sans-serif;
22 }
23 .navbar {
24     background: white;
25     padding: 10px 30px;
26     border-radius: 10px;
27     box-shadow: 0px 0px 10px rgba(0,0,0,0.05);
28     margin-bottom: 20px;
29     display: flex;
30     justify-content: space-between;
31 }
32 .nav-title {
33     font-size: 20px;
34     font-weight: bold;
35 }
36 .hero {
37     background: linear-gradient(135deg, #d9ffdb 45%, #e0ffff 55%, #e0ffff 100%);
38     padding: 10px 30px;
39     border-radius: 10px;
40     text-align: center;
41     margin-bottom: 20px;
42 }
43 .result-card {
44     background: white;
45     padding: 10px;
46     border-radius: 10px;
47     box-shadow: 0px 0px 10px rgba(0,0,0,0.05);
48 }
49 </style>
50 <script>
51 st.markdown(css, unsafe_allow_html=True)
52 # Navbar
53 st.markdown(
54     """
55     <div class="navbar">
56         <div class="nav-title">🌿 Plant Doctor</div>
57     </div>
58     """
59     , unsafe_allow_html=True
60 )
61 # Hero
62 st.markdown(
63     """
64     <div class="hero">
65         <h1>Identify Plant Diseases Instantly</h1>
66         <p>قم برفع صورة ورقة النبات وسيتم التعرف عليها</p>
67     </div>
68     """
69     , unsafe_allow_html=True
70 )
71 model = load_model("D:\saved_models\my_model.h5")
72 CLASS_NAMES = [
73     "Apple__Apple_scorch",
74     "Apple__Black_rot",
75     "Apple__Cedar_apple_rust",
76     "Apple__healthy",
77     "Blueberry__healthy",
78     "Cherry_(including_sour)__Powdery_mildew",
79     "Cherry_(including_sour)__healthy",
80     "Corn_(maize)__Cercospora_leaf_spot_Gray_leaf_spot",
81     "Corn_(maize)__Common_rust",
82     "Corn_(maize)__Northern_leaf_blight",
83     "Corn_(maize)__healthy",
84     "Grape__Black_rot",
85     "Grape__Leaf_blight_(Esca)",
86     "Grape__Leaf_blight_(Isariopsis_Leaf_Spot)",
87     "Grape__healthy",
88     "Orange__Huanglongbing_(Citrus_greening)",
89     "Peach__Bacterial_spot",
90     "Peach__healthy",
91     "Pepper,_bell__Bacterial_spot",
92     "Pepper,_bell__healthy",
93     "Potato__Early_blight",
94     "Potato__Late_blight",
95     "Potato__healthy",
96     "Raspberry__healthy",
97     "Soybean__healthy",
98     "Squash__Powdery_mildew",
99     "Strawberry__Leaf_scorch",
100     "Strawberry__healthy",
101     "Tomato__Bacterial_spot",
102     "Tomato__Early_blight",
103     "Tomato__Late_blight",
104     "Tomato__Leaf_Mold",
105     "Tomato__Septoria_leaf_spot",
106     "Tomato__Spider_mites_Two-spotted_spider_mite",
107     "Tomato__Target_Spot",
108     "Tomato__Tomato_Yellow_Leaf_Curl_Virus",
109     "Tomato__Tomato_mosaic_virus",
110     "Tomato__healthy"
111 ]
112 # Prediction
113 def predict(img):
114     img = img.resize((256, 256))
115     arr = np.array(img) / 255.0
116     arr = np.expand_dims(arr, 0)
117     pred = model.predict(arr)
118     idx = np.argmax(pred)
119     conf = np.max(pred)
120     row_name = CLASS_NAMES[idx]
121     name = row_name.replace("_", " ").replace(" ", "-").replace("healthy", "سليم")
122     return name, conf
123 # UI
124 left, right = st.columns(2)
125 with left:
126     st.header("📷 Upload Image")
127     upl = st.file_uploader("Choose a leaf image", type=["jpg", "jpeg", "png"])
128     if upl:
129         img = Image.open(upl)
130         st.image(img, caption="Uploaded Image", use_column_width=True)
131 with right:
132     st.header("🔍 Diagnosis")
133     if upl:
134         name, conf = predict(img)
135         # مخرجات النموذج
136         data = treatments.get(name, None)
137         desc = "وصف"
138         treat = "علاج"
139         prev = "الوقاية"
140         if data:
141             desc = data["description"]
142             treat = data["treatment"]
143             prev = data["prevention"]
144         st.markdown(
145             """
146             <div class="result-card">
147                 <div style="color:#1b4d32;">{name}</div>
148                 <div>
149                     <div>Confidence: {conf*100:.2f}%</div>
150                     <div>Description: {desc}</div>
151                     <div>Treatment: {treat}</div>
152                     <div>Prevention: {prev}</div>
153                 </div>
154             </div>
155             """
156             , unsafe_allow_html=True
157         )
158     st.warning("⚠️ استشارة الطبيب قبل استخدام")
159 else:
160     st.info("Upload an image to see the prediction.")
161 # Footer
162 st.markdown(
163     """
164     <div style="text-align:center; margin-top:50px; color:#777;">
165         Powered by Deep Learning 🚀
166     </div>
167     """
168     , unsafe_allow_html=True
169 )

```

Figure 5.3:(Model Deployment and Prediction Interface Implementation)

5.3 Results and Discussion

The performance of the Plant Disease Recognition System was evaluated using different plant leaf images from multiple disease categories. The trained CNN model demonstrated strong classification capabilities and achieved high prediction accuracy during testing.

The training and validation accuracy curves indicate that the model learned the disease patterns effectively throughout the training process. At the same time, the loss curves showed a continuous decrease, which reflects the stability of the learning process and the ability of the model to generalize on unseen data.

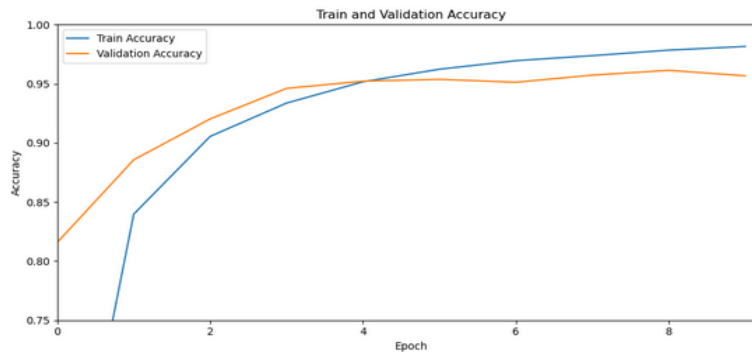


Figure 5.4: (Training and Validation Accuracy)

The accuracy graph shows that both training and validation accuracy increased gradually during training. The model achieved a final training accuracy of 98.16% and a test accuracy of 95.67%. The small gap between the curves indicates that the model achieved good generalization performance and did not suffer significantly from overfitting.

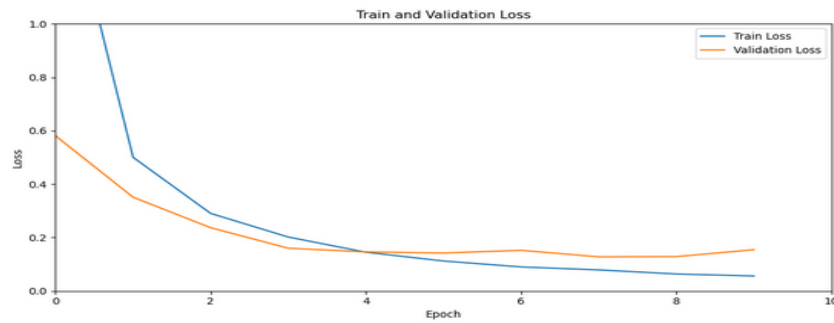


Figure 5.5: (Training and Validation Loss)

optimized its parameters and improved prediction performance over successive epochs.

The developed system was further tested using different plant leaf images through the Streamlit interface. The system successfully identified plant diseases and provided confidence scores along with treatment and prevention recommendations. The model achieved a precision score of 95.67% and a recall score of 95.67%, demonstrating its effectiveness in recognizing plant diseases across different categories.

Treatment Recommendation System

The system includes a recommendation feature that provides users with disease-related information after prediction.

The recommendation system retrieves data from a JSON file containing:

- o Disease description.
- o Treatment methods.
- o Prevention recommendations..

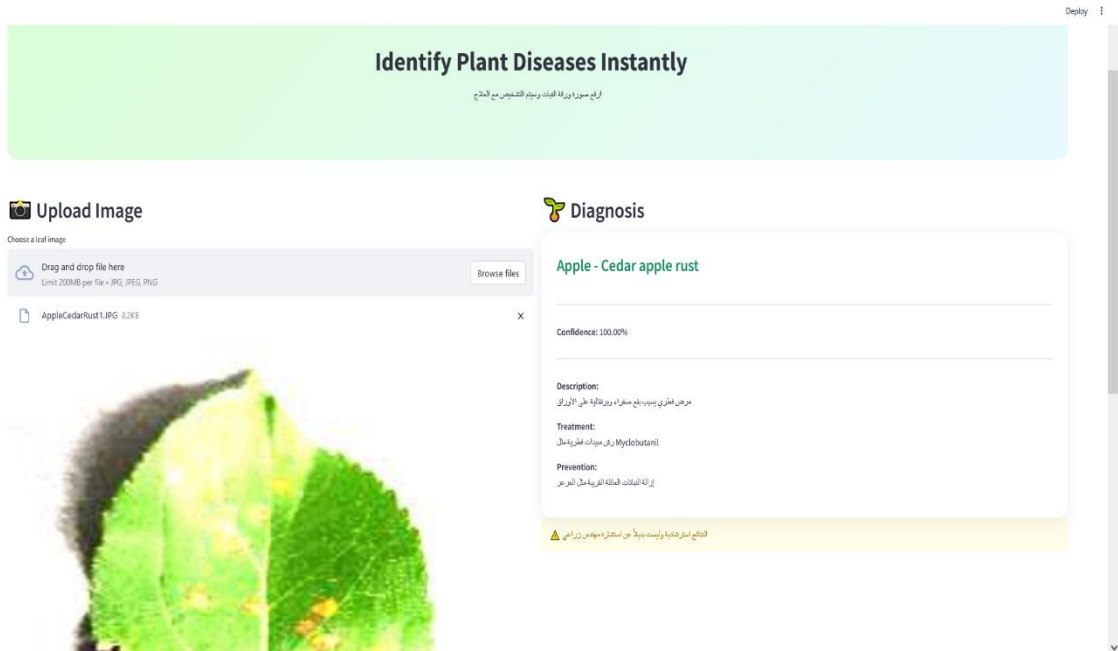


Figure 5.6: (Treatment Recommendation System)

System Features

Key Features of the System

- Automatic detection of plant diseases using a CNN-based deep learning model.
- Fast and accurate prediction with confidence scores.
- User-friendly interface for image upload and result display.
- Disease descriptions, treatment suggestions, and prevention recommendations.
- Integration of TensorFlow/Keras with Streamlit for efficient disease classification.

Chapter 6

Testing

6.1 Unit Testing

Unit testing was conducted to verify that each module of the Plant Disease Recognition System functions correctly.

Module	Expected Result	Status
Image Upload	Image uploaded successfully	Pass
Image Preprocessing	Image resized and normalized correctly	Pass
CNN Prediction	Disease predicted correctly	Pass
Confidence Score	Confidence displayed successfully	Pass
Treatment Recommendation	Treatment information retrieved correctly	Pass
User Interface	Results displayed successfully	Pass

Table 6.1(Unit Testing)

The testing results indicate that all individual modules operated successfully and produced the expected outputs.

6.2 Integration Testing

Integration testing was performed to ensure that all system components work together correctly.

The integration workflow consists of:

1. Uploading a plant leaf image.
2. Preprocessing the image.
3. Predicting the disease using the CNN model.
4. Retrieving treatment information.
5. Displaying the final results.

Overall System Architecture of the Plant Disease Recognition System

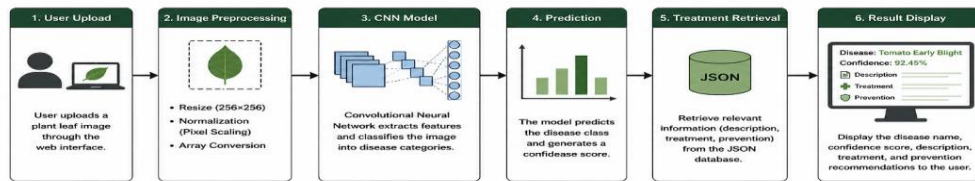


Figure 6.1: (System Integration Workflow)

Scenario	Expected Result	Status
Diseased Leaf Image	Correct disease prediction	Pass
Healthy Leaf Image	Healthy class displayed	Pass

Scenario	Expected Result	Status
Treatment Retrieval	Treatment information displayed	Pass
Final Output Display	Complete results displayed successfully	Pass

Table 6.2(Integration Testing)

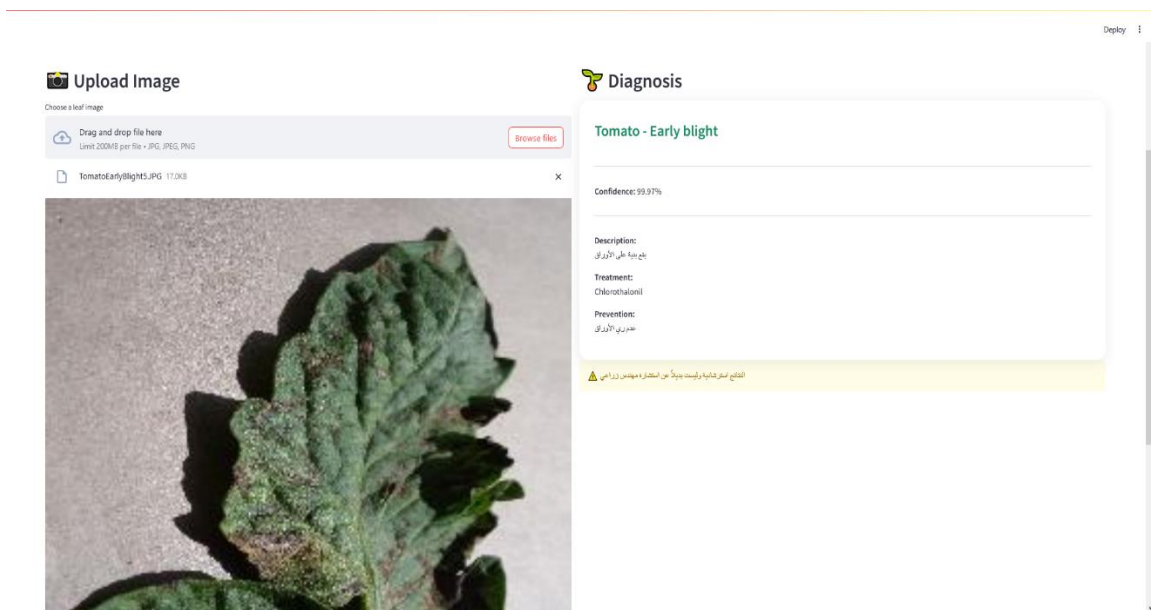


Figure 6.2: (Integrated System Output)

The integration testing results confirm that all modules communicate effectively and operate as a complete system. The application successfully processes plant leaf images, predicts diseases, retrieves recommendations, and displays the results through the user interface.

Conclusion

This project presented the design and implementation of a Plant Disease Recognition System based on deep learning techniques. The primary objective of the system was to assist users in identifying plant diseases automatically through image analysis and to provide useful recommendations for treatment and prevention.

A Convolutional Neural Network (CNN) model was developed and trained using a dataset containing images of healthy and diseased plant leaves. The model learned to extract important visual features from the images and classify them into the appropriate disease categories. Image preprocessing techniques, including resizing and normalization, were applied to improve model performance and ensure consistent input data.

To make the system practical and accessible, a user-friendly web application was developed using Streamlit. The application enables users to upload plant leaf images and receive instant predictions along with confidence scores. In addition, the system provides disease descriptions, treatment methods, and prevention guidelines through an integrated recommendation module. This feature enhances the usefulness of the application by offering not only diagnosis but also guidance for disease management.

The implementation and testing phases demonstrated that the developed system operates successfully and achieves its intended objectives. Unit testing verified the functionality of individual modules, while integration testing confirmed the proper interaction between all system components. The results showed that the model is capable of accurately recognizing

Future Work

The plant disease detection system based on deep learning performs well, but it can still be improved in several ways.

First, the dataset should be expanded to include more plant types and disease categories to improve accuracy and generalization in real-world conditions.

Second, the model can be enhanced by using advanced architectures such as transfer learning with ResNet, EfficientNet, or MobileNet, which can improve performance and training speed.

The system can also be developed into a mobile application to allow real-time disease detection in the field using smartphone cameras.

Another improvement is deploying the system on cloud platforms to make it more scalable and easier to update.

In addition, the recommendation part can be strengthened by providing more detailed treatment advice, including pesticide use, disease severity, and location-based solutions.

Explainable AI techniques can also be added to make predictions more transparent and help users understand the results.

Finally, improving the user interface with better design, interactivity, and multi-language support would enhance usability.

Overall, these improvements can make the system more accurate, practical, and useful for modern agriculture.

References

1. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
Available at: <https://www.deeplearningbook.org/>
2. Chollet, F. (2017). *Deep Learning with Python*. Manning Publications.
3. TensorFlow Documentation. (2024). *TensorFlow Guide*.
Available at: <https://www.tensorflow.org/>
4. Keras Documentation. (2024). *Keras API Reference*.
Available at: <https://keras.io/>
5. Streamlit Documentation. (2024). *Streamlit: Build Web Apps for Machine Learning*.
Available at: <https://docs.streamlit.io/>
6. PlantVillage Dataset. (2023). *Plant Disease Image Dataset*.
Available at: <https://www.kaggle.com/datasets/emmarex/plantdisease>
7. Mohanty, S. P., Hughes, D. P., & Salathé, M. (2016). *Using Deep Learning for Image-Based Plant Disease Detection*. *Frontiers in Plant Science*.
8. OpenCV Documentation. (2024). *OpenCV Image Processing Library*.
Available at: <https://opencv.org/>
9. Python Software Foundation. (2024). *Python Language Reference*.
Available at: <https://www.python.org/>

10. Chollet, F. (2015). *Convolutional Neural Networks for Image Classification*. (Concepts used in CNN architectures).
11. Mohanty, S. P., Hughes, D. P., & Salathé, M. (2016).
"Using Deep Learning for Image-Based Plant Disease Detection."
Frontiers in Plant Science.
<https://www.frontiersin.org/articles/10.3389/fpls.2016.01419>
12. Ferentinos, K. P. (2018).
"Deep Learning Models for Plant Disease Detection and Diagnosis."
Computers and Electronics in Agriculture.
<https://doi.org/10.1016/j.compag.2018.01.009>
13. Too, E. C., Yujian, L., Njuki, S., & Yingchun, L. (2019).
"A Comparative Study of Fine-Tuning Deep Learning Models for Plant Disease Identification."
Computers and Electronics in Agriculture.
<https://doi.org/10.1016/j.compag.2018.03.032>
14. Tan, M., & Le, Q. (2019).
"EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks."
<https://arxiv.org/abs/1905.11946>
15. He, K., Zhang, X., Ren, S., & Sun, J. (2016).
"Deep Residual Learning for Image Recognition (ResNet)."
<https://arxiv.org/abs/1512.03385>
16. Howard, A. et al. (2017).
"MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications."
<https://arxiv.org/abs/1704.04861>

17. Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012).
"ImageNet Classification with Deep Convolutional Neural Networks."
<https://doi.org/10.1145/3065386>
18. Shorten, C., & Khoshgoftaar, T. (2019).
"A Survey on Image Data Augmentation for Deep Learning."
<https://doi.org/10.1186/s40537-019-0197-0>

Appendices: GitHub Link

- <https://github.com/Zn634677/Agri-AI-Intelligent-Plant-Disease-Recognition-System-deep-learning>

نظام ذكي لاكتشاف وتشخيص أمراض النباتات بالاعتماد على تقنيات التعلم العميق

يهدف هذا المشروع إلى تطوير نظام ذكي للتعرف على أمراض النباتات باستخدام تقنيات التعلم العميق، وذلك للمساهمة في تحسين الإنتاج الزراعي وتقليل الخسائر الناتجة عن الأمراض النباتية. وتمثل أمراض النباتات أحد أبرز التحديات التي تواجه القطاع الزراعي، حيث يؤدي التأخر في اكتشافها إلى انخفاض الإنتاجية وزيادة التكاليف الاقتصادية.

يعتمد النظام المقترح على الشبكات العصبية التلافيفية لتحليل صور أوراق النباتات وتصنيفها إلى نباتات سليمة أو مصابة بأمراض مختلفة بدقة عالية. وقد تم استخدام مجموعة بيانات التي تحتوي على آلاف الصور المصنفة مسبقاً لتدريب النموذج وتقييمه. كما تم PlantVillage تطبيق عدد من تقنيات معالجة الصور، مثل تغيير حجم الصور، وتطبيع البيانات، وتعزيز البيانات، بهدف تحسين أداء النموذج وتقليل مشكلة الإفراط في التخصيص.

بالاعتماد على مجموعة من الأدوات والمكتبات Python تم تطوير النظام باستخدام لغة البرمجة ويتكون النظام من أربع وحدات OpenCV و Keras و TensorFlow المتخصصة، من أبرزها رئيسية تشمل واجهة المستخدم، ووحدة معالجة الصور، ونموذج التصنيف، ووحدة عرض النتائج. ويتيح النظام للمستخدم رفع صورة لورقة النبات ليقوم بتحليلها وعرض التشخيص المناسب مع درجة الثقة في النتيجة خلال وقت قصير.

أظهرت النتائج التجريبية كفاءة النموذج المقترح، حيث حقق دقة تدريب بلغت 98.16% ودقة اختبار بلغت 95.67%، مما يؤكد فعالية استخدام تقنيات التعلم العميق في التعرف على أمراض النباتات. كما أظهرت النتائج قدرة النظام على تقديم تنبؤات دقيقة وموثوقة لمختلف فئات الأمراض النباتية.

يسهم النظام في دعم المزارعين والمهندسين الزراعيين من خلال توفير أداة سريعة وسهلة الاستخدام للكشف المبكر عن الأمراض النباتية، مما يساعد على اتخاذ الإجراءات العلاجية المناسبة وتقليل الاستخدام غير الضروري للمبيدات الزراعية. ويوصى مستقبلاً بتوسيع قاعدة البيانات المستخدمة وتطوير النظام ليعمل بكفاءة أكبر في البيئات الحقلية الحقيقية ذات ظروف الإضاءة والخلفيات المتغيرة.